Password Strength Analyzer & Secure Password Generator

IBM Cybersecurity Bootcamp - Project Report

Name: _____ Date: 26 October 2026

1. Introduction

Weak and reused passwords remain one of the most common causes of account compromise. Attackers rely on brute-force attacks, dictionary attacks and lists of leaked credentials. This project is a command-line tool that evaluates how strong a password is, explains why, and helps users create strong passwords and passphrases.

2. Problem Statement

Users often choose short, predictable passwords (for example 'password123' or 'qwerty') and do not know how quickly these can be cracked. There is a need for a simple, private, offline tool that gives clear, actionable feedback.

3. Objectives

• Score password strength on a 0-100 scale with a clear rating.

• Detect common passwords and weak patterns (repeats, sequences, keyboard walks).

• Estimate brute-force crack time using entropy.

• Generate cryptographically secure passwords and passphrases.

• Keep all processing local so no password is stored or transmitted.

4. Tools and Technologies

• Python 3 (standard library only): argparse, getpass, math, re, secrets, string

• Git and GitHub for version control and submission

5. Methodology

Scoring: The score combines length (up to 40 points), character variety across lowercase, uppercase, digits and symbols (up to 28 points) and an entropy bonus (up to 32 points). Penalties of 10 points apply for repeated characters, sequences and keyboard patterns. Passwords found in a common-password list are capped at 5.

Entropy: Entropy (bits) = length x log2(size of character set). Estimated crack time assumes an offline attacker making 10 billion guesses per second and succeeding on average after trying half the search space.

Generation: Passwords are generated with Python's *secrets* module, which uses the operating system's cryptographically secure random source, unlike the *random* module, which is predictable. At least one character from each category is guaranteed, then the result is securely shuffled. Passphrases join randomly chosen words with a random number.

Privacy: Input is collected with *getpass* so it is not echoed to the screen, and nothing is written to disk or sent over a network.

6. Results

Test password	Score	Rating	Est. crack time
password	5	Very Weak	Instantly
Qwerty123	0	Very Weak	Instantly
Tr0ub4dor&3	78	Strong	~8,000 years
k9#Lm2$vQx8!pZw4	92	Very Strong	Billions of years+

Common and pattern-based passwords were correctly flagged as very weak, while long, mixed-character random passwords scored highest. Generated passwords consistently scored above 85.

7. Security Concepts Demonstrated

• Entropy and brute-force resistance

• Dictionary and pattern attacks

• CSPRNG vs. predictable PRNG

• Secure input handling and privacy by design

8. Limitations

• Entropy is an estimate and does not capture every real-world cracking technique.

• The common-password list and passphrase wordlist are small.

• No online breach database lookup.

9. Future Scope

• Integrate the Have I Been Pwned range API using k-anonymity.

• Add a GUI or web interface.

• Use larger wordlists such as SecLists and add unit tests.

10. Conclusion

The project demonstrates practical password security: how length, variety and unpredictability affect strength, and how to generate secure credentials safely. Users get clear feedback to improve their password habits, supporting a stronger overall security posture.